BTS CIEL · PROGRAMMATION C++ · COURS

Chapitre 6 — Représentation des données et opérations bit à bit

Prérequis : chapitres 1 à 5 · **Exercices :** `E06_Representation_donnees_bit_a_bit`

Objectifs

- passer du décimal au binaire et à l'hexadécimal ;
- relier un caractère à son code ASCII et décoder une trame ;
- utiliser les types entiers de taille fixe et les opérateurs bit à bit `&`, `|`, `^`, `~`, `<<`, `>>` ;
- appliquer ces notions aux masques, au masquage XOR et à l'adressage IPv4.

Dans les systèmes numériques et les réseaux, l'information circule sous forme d'**octets** : trames série, paquets réseau, registres de microcontrôleur, masques de sous-réseau. Ce chapitre donne les outils pour les lire et les transformer.

Sommaire

- Chapitre 6 — Représentation des données et opérations bit à bit
  - Objectifs
  - Sommaire
  - 1. Binaire, décimal et hexadécimal
    - 1.1 Le binaire
    - 1.2 L'hexadécimal
    - 1.3 En C++ : constantes et affichage
  - 2. Caractères et codes ASCII
  - 3. Les types entiers de taille fixe
  - 4. Les opérateurs bit à bit
    - 4.1 Le complément `~`
    - 4.2 Les décalages `<<` et `>>`
  - 5. Les masques binaires
  - 6. Application : masquage XOR avec une clé répétée
    - 6.1 Principe
    - 6.2 Démasquer un message
    - 6.3 Masquer puis démasquer
  - 7. Application : adressage IPv4
  - Erreurs fréquentes
  - Synthèse

1. Binaire, décimal et hexadécimal

1.1 Le binaire

Chaque position d'un nombre binaire représente une puissance de 2.

Position du bit	7	6	5	4	3	2	1	0
Poids	128	64	32	16	8	4	2	1
Bits de 13	0	0	0	0	1	1	0	1

13 = 8 + 4 + 1, soit `00001101` sur un octet. Le bit 0 est le bit **de poids faible**, le bit 7 celui **de poids fort**.

1.2 L'hexadécimal

L'hexadécimal utilise les chiffres `0` à `9` puis `A` (10) à `F` (15). Comme 16 = 2^4, **un chiffre hexadécimal correspond exactement à 4 bits** : un octet s'écrit toujours avec **deux** chiffres. C'est pourquoi les analyseurs de trames affichent les octets en hexadécimal.

Décimal	Hexadécimal	Binaire	Décimal	Hexadécimal	Binaire
0	0	0000	8	8	1000
1	1	0001	9	9	1001
2	2	0010	10	A	1010
3	3	0011	11	B	1011
4	4	0100	12	C	1100
5	5	0101	13	D	1101
6	6	0110	14	E	1110
7	7	0111	15	F	1111

- **Hexadécimal → décimal :** chiffre de gauche × 16 + chiffre de droite. `0x2F` = 2 × 16 + 15 = **47** ; `0xC0` = **192** ; `0xFF` = **255**.
- **Hexadécimal ⚿ binaire :** chaque chiffre donne 4 bits. `0xC0` = `1100 0000`.
- **Décimal → hexadécimal :** on divise par 16 ; le quotient donne le chiffre de gauche, le reste celui de droite. 200 = 12 × 16 + 8, soit `0xC8`.

1.3 En C++ : constantes et affichage

```cpp
#include <iomanip>
#include <iostream>

int main() {
    int a{47};            // decimal
    int b{0x2F};          // hexadecimal : prefixe 0x
    int c{0b00101111};    // binaire : prefixe 0b
    // a, b et c ont exactement la meme valeur
    std::cout << (a == b && b == c) << '\n';

    std::cout << std::hex << std::uppercase << 200 << '\n';   // C8
    std::cout << std::dec << 200 << '\n';                      // retour au decimal

    // Un octet sur deux chiffres : 0x0A plutot que 0xA
    std::cout << "0x" << std::hex << std::setw(2) << std::setfill('0')
              << 10 << std::dec << '\n';

    return 0;
}
```

Résultat :

```
1
C8
200
0x0A
```

- `std::hex`, `std::dec` et `std::uppercase` **restent actifs** sur le flux jusqu'à ce qu'on les change.
- `std::setw(2)` (en-tête `<iomanip>`) fixe la largeur du **prochain** affichage seulement ; `std::setfill('0')` complète avec des zéros.

2. Caractères et codes ASCII

Un `char` stocke un **nombre** : le code du caractère. Dans le code **ASCII**, utilisé par la plupart des protocoles textuels, les caractères courants ont un code entre 0 et 127.

Caractère	Décimal	Hexadécimal
espace	32	0x20
`/`	47	0x2F
`0` à `9`	48 à 57	0x30 à 0x39
`:`	58	0x3A
`=`	61	0x3D
`A` à `Z`	65 à 90	0x41 à 0x5A
`a` à `z`	97 à 122	0x61 à 0x7A

Les chiffres sont consécutifs à partir de `'0'` (`0x30`). Une minuscule vaut la majuscule correspondante + 32.

```cpp
#include <cstdint>
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::cout << static_cast<int>('A') << ' ' << ('7' - '0') << '\n';   // code ; valeur du
chiffre

    // Decoder une trame textuelle relevee en hexadecimal
    std::vector<std::uint8_t> trame{0x32, 0x2F, 0x31, 0x35, 0x2F, 0x30};
    std::string texte{};
    for (std::uint8_t octet : trame) {
        texte += static_cast<char>(octet);
    }
    std::cout << texte << '\n';

    return 0;
}
```

Résultat :

```
65 7
2/15/0
```

> **À ne pas confondre :** le **caractère** `'7'` (code 55, `0x37`) et l'**entier** 7. Une trame textuelle envoie la valeur 25 sous la forme de deux octets : `0x32` (`'2'`) et `0x35` (`'5'`).

3. Les types entiers de taille fixe

La taille de `int` dépend de la plateforme. Pour manipuler des octets ou des registres, on utilise les types de `<cstdint>`, de taille **garantie** :

Type	Taille	Valeurs
`std::uint8_t`	8 bits	0 à 255
`std::uint16_t`	16 bits	0 à 65 535
`std::uint32_t`	32 bits	0 à 4 294 967 295 (adresse IPv4)
`std::int8_t`, `std::int16_t`, `std::int32_t`	8, 16, 32 bits	Versions signées

> **Piège :** `std::uint8_t` est en pratique un `unsigned char` : `std::cout` l'affiche comme un **caractère**. Pour afficher sa valeur, on écrit `static_cast<unsigned int>(octet)`.

4. Les opérateurs bit à bit

Ils agissent **bit par bit** sur des entiers, de préférence non signés.

A	B	A `&` B (ET)	A `|` B (OU)	A `^` B (OU exclusif, XOR)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

```
  0101 (5)      0101 (5)      0101 (5)
& 0011 (3)    | 0011 (3)    ^ 0011 (3)
= 0001 (1)    = 0111 (7)    = 0110 (6)
```

À retenir : `(a ^ k) ^ k == a`. Appliquer deux fois le même XOR redonne la valeur de départ. C'est le principe du masquage et du démasquage (§6).

```cpp
#include <iostream>

int main() {
    unsigned int a{5};
    unsigned int b{3};

    std::cout << (a & b) << ' ' << (a | b) << ' ' << (a ^ b) << '\n';

    return 0;
}
```

Résultat :

```
1 7 6
```

4.1 Le complément `~`

`~` inverse **tous** les bits de la valeur. Le résultat dépend donc de la **largeur** du type. De plus, avant le calcul, C++ convertit les petits types (`std::uint8_t`, `char`…) en `int` : c'est la **promotion entière**. `~` agit donc sur 32 bits, et non sur 8.

```cpp
#include <cstdint>
#include <iostream>

int main() {
    std::uint8_t valeur{0b00001111};

    // ~valeur est calcule sur un int (32 bits) : 0xFFFFFFF0
    // La conversion en uint8_t ne conserve que les 8 bits de poids faible : 0xF0
    std::uint8_t complement{static_cast<std::uint8_t>(~valeur)};

    std::cout << static_cast<unsigned int>(complement) << '\n';

    return 0;
}
```

Résultat :

```
240
```

La représentation sur huit bits est :

```
Valeur       : 00001111
Complement   : 11110000
```

4.2 Les décalages `<<` et `>>`

- `<<` décale les bits vers la gauche et insère des 0 à droite. Pour un non signé, décaler de n revient à multiplier par 2^n (tant que le résultat tient dans le type).
- `>>` décale vers la droite. Pour un non signé, décaler de n revient à diviser par 2^n (division entière).

```
0101 << 1 = 1010     (5 -> 10)
1100 >> 1 = 0110     (12 -> 6)
```

> **Interdit :** un décalage d'un nombre **négatif** de positions, ou d'un nombre **supérieur ou égal** à la largeur du type (`x << 32` sur 32 bits), est un comportement indéfini.

5. Les masques binaires

Un **masque** est une valeur dont les bits à 1 désignent les bits sur lesquels on agit. Pour viser le bit n : `1u << n`.

Opération	Écriture	Effet sur les autres bits
Tester le bit n	`(etat & (1u << n)) != 0`	Aucun
Activer le bit n	`etat |= (1u << n);`	Conservés
Désactiver le bit n	`etat &= ~(1u << n);`	Conservés
Inverser le bit n	`etat ^= (1u << n);`	Conservés
Extraire un octet	`(valeur >> 8) & 0xFF`	Seuls 8 bits gardés

```cpp
#include <iostream>

int main() {
    constexpr unsigned int BIT_ALIMENTATION{1u << 0};   // 0001
    constexpr unsigned int BIT_CAPTEUR{1u << 1};        // 0010
    constexpr unsigned int BIT_ALARME{1u << 2};         // 0100

    unsigned int etat{0};

    etat |= BIT_ALIMENTATION;                 // 0001
    etat |= BIT_ALARME;                       // 0101

    if ((etat & BIT_ALARME) != 0u) {
        std::cout << "Alarme active\n";
    }

    etat &= ~BIT_ALIMENTATION;                // 0100
    etat ^= BIT_CAPTEUR;                      // 0110

    std::cout << "Etat final : " << etat << '\n';

    return 0;
}
```

Résultat :

```
Alarme active
Etat final : 6
```

> **Ne pas confondre :** `&&`, `||` et `!` combinent des **conditions** et produisent un `bool`. `&`, `|`, `^`, `~`, `<<` et `>>` transforment les **bits** d'un entier et produisent un entier.

> **Attention :** priorités. `etat & masque != 0` est évalué comme `etat & (masque != 0)`. On écrit toujours `(etat & masque) != 0`.

6. Application : masquage XOR avec une clé répétée

6.1 Principe

Certains protocoles masquent les données transmises : c'est le cas du protocole WebSocket, qui utilise une clé de quatre octets. Chaque octet des données utiles (*payload*) est combiné par XOR avec un octet de la **clé**. La clé étant plus courte que le message, elle est **répétée** : l'octet d'indice `i`, **compté depuis zéro au début des données utiles**, utilise l'octet de clé d'indice `i % tailleCle`.

```
Indice des donnees : 0   1   2   3   4   5   6   7 ...
Indice de cle      : 0   1   2   3   0   1   2   3 ...
```

La relation `brut ^ cle = masque` est réversible : `masque ^ cle = brut`. On masque et on démasque donc avec **la même opération** et **la même clé**.

> **Attention :** une documentation qui parle du « 14e octet » compte généralement à partir de 1 : en C++, cet octet a l'indice **13**.

6.2 Démasquer un message

```cpp
#include <array>
#include <cstddef>
#include <cstdint>
#include <iostream>
#include <string>

int main() {
    const std::array<std::uint8_t, 4> cle{0x10, 0x20, 0x30, 0x40};
    const std::array<std::uint8_t, 5> masque{0x53, 0x69, 0x75, 0x0C, 0x31};

    std::string message{};
    for (std::size_t i = 0; i < masque.size(); i++) {
        std::uint8_t brut{static_cast<std::uint8_t>(masque[i] ^ cle[i % cle.size()])};
        message += static_cast<char>(brut);
    }

    std::cout << message << '\n';

    return 0;
}
```

Résultat :

```
CIEL!
```

Par exemple, `0x53 ^ 0x10 = 0x43`, code ASCII de `C`. L'indice 4 réemploie le premier octet de clé, car `4 % 4` vaut 0.

6.3 Masquer puis démasquer

```cpp
#include <cstddef>
#include <cstdint>
#include <iomanip>
#include <iostream>
#include <vector>

int main() {
    const std::vector<std::uint8_t> cle{0x12, 0x34, 0x56, 0x78};
    const std::vector<std::uint8_t> message{0x4F, 0x4B, 0x3A, 0x34, 0x32};   // "OK:42"

    std::vector<std::uint8_t> masque{};
    for (std::size_t i = 0; i < message.size(); i++) {
        masque.push_back(static_cast<std::uint8_t>(message[i] ^ cle[i % cle.size()]));
    }

    std::cout << "Masque  :";
    for (std::uint8_t octet : masque) {
        std::cout << ' ' << std::hex << std::uppercase << std::setw(2)
                  << std::setfill('0') << static_cast<unsigned int>(octet);
    }
    std::cout << std::dec << '\n';

    std::cout << "Demasque : ";              // meme operation, meme cle
    for (std::size_t i = 0; i < masque.size(); i++) {
        std::cout << static_cast<char>(masque[i] ^ cle[i % cle.size()]);
    }
    std::cout << '\n';

    return 0;
}
```

Résultat :

```
Masque  : 5D 7F 6C 4C 20
Demasque : OK:42
```

Le cinquième octet (indice 4) réutilise le premier octet de la clé.

7. Application : adressage IPv4

Une adresse IPv4 est un nombre de **32 bits**, noté en quatre octets décimaux. Le **masque de sous-réseau** a ses bits à 1 sur la partie réseau. Un préfixe CIDR (`/24`) indique le nombre de ces bits. L'**adresse réseau** est le ET bit à bit entre l'adresse et le masque.

```
Adresse IP : 192.168.1.10   = 11000000.10101000.00000001.00001010
Masque     : 255.255.255.0  = 11111111.11111111.11111111.00000000
ET         : 192.168.1.0    = 11000000.10101000.00000001.00000000
```

Dans un `std::uint32_t`, le premier octet occupe les bits 31 à 24 et le dernier les bits 7 à 0. On **assemble** une adresse en décalant chaque octet à sa place puis en les combinant avec `|` ; on **extrait** un octet en le ramenant à droite avec `>>` puis en gardant 8 bits avec `& 0xFF`.

```cpp
#include <cstdint>
#include <iostream>

int main() {
    // Assembler : chaque octet est decale a sa place, puis combine avec |
    std::uint32_t adresse{(std::uint32_t{192} << 24) | (std::uint32_t{168} << 16)
                          | (std::uint32_t{1} << 8) | std::uint32_t{10}};

    // Extraire : on ramene l'octet a droite, puis on garde 8 bits
    std::uint32_t premier{(adresse >> 24) & 0xFFu};
    std::uint32_t dernier{adresse & 0xFFu};

    std::cout << std::hex << std::uppercase << adresse << std::dec << '\n';
    std::cout << premier << " ... " << dernier << '\n';

    return 0;
}
```

Résultat :

```
C0A8010A
192 ... 10
```

Piège : `192 << 24` calculé sur un `int` (signé) ne tient pas dans 31 bits : le résultat est négatif, donc faux pour une adresse. On décale toujours des **non signés** : `std::uint32_t{192} << 24`.

Erreurs fréquentes

Erreur	Correction
Confondre `'5'` (`0x35`) et `5`	`'5' - '0'` vaut 5
Afficher un `std::uint8_t` directement	`static_cast<unsigned int>(octet)`
Oublier `std::dec` après `std::hex`	La suite s'affiche en hexadécimal
`etat & masque != 0`	`(etat & masque) != 0`
Décalage de 32 bits ou plus	Traiter les cas limites à part
Décaler un `int` signé (`192 << 24`)	Décaler un non signé (`std::uint32_t`)
`~` sur un `std::uint8_t` sans conversion	`static_cast<std::uint8_t>(~valeur)`
« 14e octet » lu à l'indice 14	Indice C++ = rang − 1

Synthèse

- 1 chiffre hexadécimal = 4 bits ; 1 octet = 2 chiffres. `0x2F` = 47.
- Un caractère est un code : `'0'` = `0x30`, `'A'` = `0x41` ; valeur d'un chiffre : `c - '0'`.
- Types de taille fixe : `std::uint8_t`, `std::uint16_t`, `std::uint32_t` ; un `std::uint8_t` s'affiche après conversion en `unsigned int`.

- Masques : tester `&`, activer `|=`, désactiver `&= ~`, inverser `^=` ; parenthèses autour de `(x & masque)`.
- XOR appliqué deux fois avec la même clé redonne la valeur ; clé répétée : indice `i % tailleCle`.
- IPv4 : adresse réseau = adresse `&` masque ; assembler avec `<<` et `|`, extraire avec `>>` et `& 0xFF`, toujours sur des non signés.